

GE23131-Programming Using C-2024

Quiz navigation



Show one page at a time

Finish review

Status	Finished
Started	Monday, 23 December 2024, 5:33 PM
Completed	Tuesday, 22 October 2024, 2:52 PM
Duration	62 days 2 hours

Question **1**

Correct

Marked out of 3.00

Flag question

Write a program to input a name (as a single character) and marks of three tests as m1, m2, and m3 of a student considering all the three marks have been given in integer format.

Now, you need to calculate the average of the given marks and print it along with the name as mentioned in the output format section.

All the test marks are in integers and hence calculate the average in integer as well. That is, you need to print the integer part of the average only and neglect the decimal part.

Input format :

Line 1 : Name(Single character)

Line 2 : Marks scored in the 3 tests separated by single space.

Output format :

First line of output prints the name of the student.

Second line of the output prints the average mark.

Constraints

Marks for each student lie in the range 0 to 100 (both inclusive)

Sample Input 1 :

A
3 4 6

Sample Output 1 :

A
4

Sample Input 2 :

T

7 3 8

Sample Output 2 :

T
6

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int m1,m2,m3;
6     char a;
7     scanf("%c", &a);
8     scanf("%d %d %d", &m1,&m2,&m3);
9     printf("%c\n",a);
10    printf("%d", (m1+m2+m3)/3);
11    return 0;
12 }
```

	Input	Expected	Got	
✓	A 3 4 6	A 4	A 4	✓
✓	T 7 3 8	T 6	T 6	✓
✓	R 0 100 99	R 66	R 66	✓

Passed all tests! ✓

Question **2**

Correct

Marked out of
5.00

🚩 Flag question

Some C data types, their format specifiers, and their most common bit widths are as follows:

- *Int* ("%d"): 32 Bit integer
- *Long* ("%ld"): 64 bit integer
- *Char* ("%c"): Character type
- *Float* ("%f"): 32 bit real value
- *Double* ("%lf"): 64 bit real value

Reading

To read a data type, use the following syntax:

```
scanf("format_specifier", &val)
```

For example, to read a *character* followed by a *double*:

```
char ch;  
double d;  
scanf("%c %lf", &ch, &d);
```

For the moment, we can ignore the spacing between format specifiers.

Printing

To print a data type, use the following syntax:

```
printf("format_specifier", val)
```

For example, to print a *character* followed by a *double*:

```
char ch = 'd';  
double d = 234.432;  
printf("%c %lf", ch, d);
```

Note: You can also use *cin* and *cout* instead of *scanf* and *printf*; however, if you are taking a million numbers as input and printing a million lines, it is faster to use *scanf* and *printf*.

Input Format

Input consists of the following space-separated values: *int*, *long*, *char*, *float*, and *double*, respectively.

Output Format

Print each element on a new line in the same order it was received as input. Note that the floating point value should be correct up to 3 decimal places and the double to 9 decimal places.

Sample Input

```
3 12345678912345 a 334.23 14049.30493
```

Sample Output

```
3  
12345678912345  
a  
334.230  
14049.304930000
```

Explanation

Print *int* **3**,

followed by *long* **12345678912345**,

followed by *char* **a**,

followed by *float* **334.23**,

followed by *double* **14049.30493**.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>  
2 int main()  
3 {  
4     int i;  
5     long l;  
6     char c;  
7     float f;  
8     double d;  
9     scanf("%d %ld %c %f %lf", &i,&l,&c,&f,&d);  
10  
11     printf("%d\n", i);  
12     printf("%ld\n", l);  
13     printf("%c\n", c);  
14     printf("%.3f\n", f);  
15     printf("%.9lf\n", d);  
16     return 0;  
17 }
```

	Input	Expected	Got	
✓	3 12345678912345 a 334.23 14049.30493	3 12345678912345 a 334.230 14049.304930000	3 12345678912345 a 334.230 14049.304930000	✓

Passed all tests! ✓

Question **3**
Correct
Marked out of 7.00
[Flag question](#)

Write a program to print the [ASCII value](#) and the two adjacent characters of the given character.

Input

E

Output

69

D F

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     char ch;
5     scanf("%c",&ch);
6     printf("%d\n",ch);
7     printf("%c %c\n",ch-1,ch+1);
8     return 0;
9 }
```

Input	Expected	Got
-------	----------	-----

✓	E	69 D F	69 D F	✓
---	---	-----------	-----------	---

Passed all tests! ✓

Finish review